30 CONNECTING MONGODB TO NESTJS — TWO APPROACHES

DatabaseService (src/database/database.service.ts) already has the right SHAPE: onModuleInit() connects, onApplicationShutdown() cleans up (see theory/27-lifecycle-events.txt). What it's missing is a real driver behind connect() — right now it just flips a boolean. This file covers the two ways to make that real.

APPROACH 1 — NATIVE `mongodb` DRIVER (manual, what your file is set up for)

Install: npm install mongodb

You own the client directly. DatabaseService becomes the single place that holds the connection; every other service that needs data asks DatabaseService for it via DI, instead of each service opening its own connection.

```
import { Injectable, OnModuleInit, OnApplicationShutdown } from '@nestjs/common';
import { MongoClient, Db } from 'mongodb';

@Injectable()
export class DatabaseService implements OnModuleInit, OnApplicationShutdown {
  private client: MongoClient;
  private db: Db;

  async onModuleInit() {
    this.client = new MongoClient('mongodb://localhost:27017');
    await this.client.connect();
    this.db = this.client.db('myapp');
    console.log('DatabaseService connected');
  }

  getDb(): Db {
    return this.db; // other services call this to get collections
  }

  async onApplicationShutdown(signal?: string) {
    await this.client.close();
    console.log(`DatabaseService closing (${signal})`);
  }
}
```

Note onModuleInit is now `async` — Nest awaits lifecycle hooks that return a Promise before moving on, so the app won't start accepting requests until the connection is actually open.

A consuming service just injects DatabaseService like any other provider (theory/07-dependency-injection.txt):

```
@Injectable()
export class TaskService {
  constructor(private readonly databaseService: DatabaseService) {}

  findAll() {
    return this.databaseService.getDb().collection('tasks').find().toArray();
  }
}
```

Pros: you see exactly what's happening, no extra abstraction. Cons: no schema validation, no models — every query works with plain objects, and you're responsible for connection-string management, pooling options, etc.

APPROACH 2 — @nestjs/mongoose (idiomatic Nest, recommended for real apps)

Install: npm install @nestjs/mongoose mongoose

This does NOT replace DatabaseService's job — it IS the same onModuleInit/onApplicationShutdown wiring you already understand, just packaged as a dynamic module so you don't hand-roll it.

1. Register the connection once, globally, in AppModule:

```
import { MongooseModule } from '@nestjs/mongoose';

@Module({
  imports: [
    MongooseModule.forRoot('mongodb://localhost:27017/myapp'),
  ],
})
export class AppModule {}
```

Combine with theory/29-nestjs-config-module.txt to pull the URI from an env var instead of hardcoding it:

```
MongooseModule.forRootAsync({
  inject: [ConfigService],
  useFactory: (config: ConfigService) => ({
    uri: config.get<string>('MONGO_URI'),
  }),
}),
```

2. Define a schema (this is the ODM layer the native driver doesn't give you — it validates shape at the application level):

```
import { Schema, Prop, SchemaFactory } from '@nestjs/mongoose';
import { Document } from 'mongoose';

@Schema()
export class Task extends Document {
  @Prop({ required: true })
  title: string;

  @Prop({ default: false })
  done: boolean;
}

export const TaskSchema = SchemaFactory.createForClass(Task);
```

3. Register that schema in whichever feature module owns it:

```
@Module({
  imports: [MongooseModule.forFeature([{ name: Task.name, schema: TaskSchema }])],
})
export class TaskModule {}
```

4. Inject the model into a service the same way you inject anything else — @InjectModel is just a parameter decorator that tells Nest's DI container which provider token to resolve (theory/06-decorators.txt, theory/07-dependency-injection.txt):

```
@Injectable()
export class TaskService {
  constructor(@InjectModel(Task.name) private taskModel: Model<Task>) {}

  findAll() {
    return this.taskModel.find().exec();
  }
}
```

WHY APPROACH 2's forRoot() ISN'T MAGIC

MongooseModule.forRoot() internally implements OnModuleInit to open the connection during app startup and OnApplicationShutdown to close it during app teardown — literally the two hooks DatabaseService already implements. The manual version in Approach 1 exists so you can see that mechanism with nothing hidden; forRoot() is that same mechanism, battle-tested, plus pooling, auto-reconnect, and schema validation on top.

DECISION RULE

Still learning how Nest's lifecycle/DI wiring works → keep extending DatabaseService by hand (Approach 1).

Building an app you intend to keep → @nestjs/mongoose (Approach 2). Don't hand-roll a client lifecycle that a maintained module already solves.

Either way, the connection string belongs in an env var, not hardcoded — see theory/28-environment-variables.txt and theory/29-nestjs-config-module.txt.

See also: theory/27-lifecycle-events.txt, theory/07-dependency-injection.txt, theory/03-service.txt, theory/01-module.txt